

Arrays: muchos valores, una sola variable

Crear, recorrer, modificar, ordenar y buscar en arreglos



CONTENIDO

De la idea básica a los métodos

01 Introducción a los arrays

02 Trabajar con arrays

03 Añadir elementos

04 Eliminar elementos

05 Copiar parte de un array

06 Ordenar arrays

07 Arrays como pila

08 Arrays como cola

09 Buscar elementos

10 Copiar dentro del array

11 Convertir a cadena

01 · INTRODUCCIÓN

Sin *arrays*, cada artículo del carrito necesita su propia variable

✗ Variables sueltas

```
const articulo1 = "Tocadiscos";  
const articulo2 = "Altavoz";  
const articulo3 = "Preamplificador";  
const articulo4 = "Cables de altavoz";  
// ¿y si el usuario añade un 5.º?
```

✓ Un array

```
const carrito = [  
  "Tocadiscos",  
  "Altavoz",  
  "Preamplificador",  
  "Cables de altavoz"  
];
```

Un array es una **lista ordenada de valores** dentro de una sola variable: crece cuando hace falta y se puede recorrer, ordenar o buscar sin repetir código.

01 · INTRODUCCIÓN

Cada elemento ocupa una posición: el índice, que empieza en 0



carrito.length vale 4 · el último índice es siempre **length - 1**

```
const primero = carrito[0];           // "Tocariscos"
const ultimo  = carrito[carrito.length - 1]; // "Cables de altavoz"
console.log(carrito[7]);               // undefined (no existe)
```

Un *array* admite tipos distintos y crece según lo necesites

```
const valores = [  
  "Ana García",      // cadena  
  22,                // número  
  true,              // booleano  
  { ciudad: "Gijón" }, // objeto  
  [1, 2, 3]           // otro array  
];  
valores.push("nuevo"); // ahora length = 6
```

Tipado dinámico

No se declara el tipo de los elementos, a diferencia de Java.

Tamaño dinámico

No se reserva la longitud: se añade y se quita en cualquier momento.

const no congela el array

La variable no se puede reasignar, pero su contenido sí puede cambiar.

01 · INTRODUCCIÓN

Un *array* de *arrays* forma una tabla: `carrito[filas][columna]`

	[0] nombre	[1] precio	[2] cantidad
[0]	Tocadiscos	200	1
[1]	Altavoz	400	2
[2]	Preamplificador	80	1
[3]	Cables de altavoz	20	2

`carrito[1][1]` → fila 1, columna 1 → 400

```
const carrito = [  
  ["Tocadiscos", 200, 1],  
  ["Altavoz", 400, 2],  
  ["Preamplificador", 80, 1],  
  ["Cables de altavoz", 20, 2]  
];  
  
carrito[1][1]; // 400  
carrito[3][0]; // "Cables de altavoz"
```

Dos formas de crear un *array*: literal o constructor

CÓDIGO	RESULTADO	COMENTARIO
<code>const vacio = [];</code>	<code>[]</code>	Literal vacío (lo habitual)
<code>const nombres = ["Ana", "Luis"];</code>	<code>["Ana", "Luis"]</code>	Literal con valores
<code>new Array()</code>	<code>[]</code>	Constructor sin argumentos
<code>new Array(20)</code>	<code>20 huecos vacíos</code>	¡Un solo número = longitud!
<code>new Array(3, 5)</code>	<code>[3, 5]</code>	Varios números = elementos
<code>Array.of(20)</code>	<code>[20]</code>	ES6: evita la trampa anterior

Arrays de objetos y dos maneras de recorrerlos

contactos.js

```
const contactos = [
  { nombre: "Ana",  apellido: "Pérez" },
  { nombre: "Luis", apellido: "Díaz" },
  { nombre: "Eva",  apellido: "Ruiz" }
];

contactos[1].nombre; // "Luis"
```

recorrido.js

```
// for clásico: controlas el índice
for (let i = 0; i < contactos.length; i++) {
  console.log(i, contactos[i].nombre);
}

// for...of (ES6): recorres los valores
for (const contacto of contactos) {
  console.log(contacto.nombre);
}
```

Mapa de métodos: elige según lo que quieras hacer

Añadir

```
push()  
unshift()  
splice()
```

modifica el array

Eliminar

```
pop()  
shift()  
splice()
```

modifica el array

Ordenar

```
sort()  
reverse()
```

modifica el array

Copiar dentro

```
copyWithin()
```

modifica el array

Extraer y combinar

```
slice()  
concat()
```

no lo modifica

Buscar

```
indexOf()  
lastIndexOf()  
includes()  
find() · findIndex()
```

no lo modifica

Convertir a texto

```
join()  
toString()  
toLocaleString()
```

no lo modifica

Recorrer y transformar

```
forEach() · map()  
filter() · reduce()
```

en la UT 02.2

03 · AÑADIR ELEMENTOS

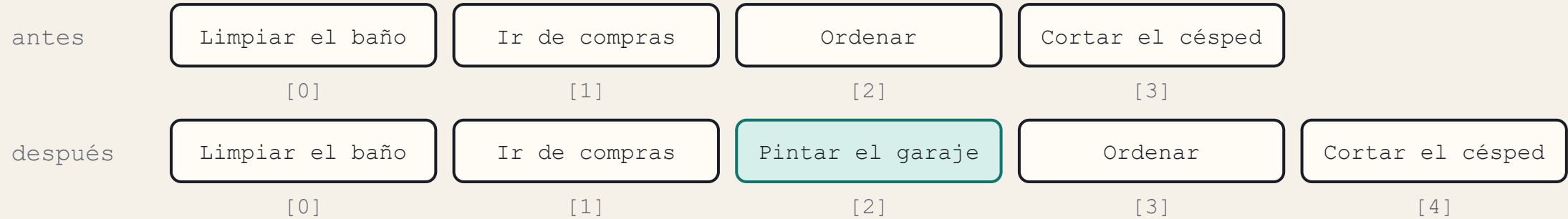
push() añade al final; unshift(), al principio



```
const tareas = ["Ir de compras"];
tareas.push("Ordenar");           // → 2 (devuelve la nueva longitud)
tareas.unshift("Limpiar el baño"); // → 3 (desplaza los índices)
tareas.push("Cortar el césped", "Regar"); // → 5 (acepta varios)
console.log(tareas);
// ["Limpiar el baño", "Ir de compras", "Ordenar", "Cortar el césped", "Regar"]
```

03 · AÑADIR ELEMENTOS

splice(posición, 0, ...nuevos) inserta donde quieras



```
const tareas = ["Limpiar el baño", "Ir de compras", "Ordenar", "Cortar el césped"];
tareas.splice(
  2,           // índice donde insertar
  0,           // cuántos eliminar (0 = ninguno)
  "Pintar el garaje" // elemento(s) a insertar
);
```

04 · ELIMINAR ELEMENTOS

pop() quita el último; shift() quita el primero

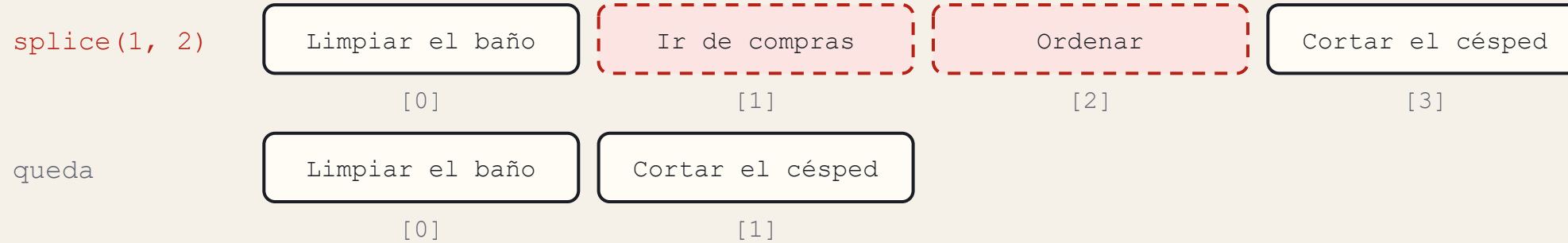


```
const tareas = ["Limpiar el baño", "Ir de compras", "Ordenar"];
const ultima = tareas.pop(); // "Ordenar"
const primera = tareas.shift(); // "Limpiar el baño"
console.log(tareas); // ["Ir de compras"]
[].pop(); // undefined (array vacío)
```

Los dos **devuelven el elemento eliminado** y modifican el array original.

04 · ELIMINAR ELEMENTOS

splice(posición, cuántos) elimina un tramo del medio



```
const tareas = ["Limpiar el baño", "Ir de compras", "Ordenar", "Cortar el césped"];
const eliminadas = tareas.splice(1, 2);
console.log(eliminadas); // ["Ir de compras", "Ordenar"]
console.log(tareas);     // ["Limpiar el baño", "Cortar el césped"]
// Con más argumentos también sustituye: tareas.splice(1, 1, "Regar")
```

05 · COPIAR PARTE DE UN ARRAY

slice(inicio, fin) copia un tramo sin tocar el original

```
const tareas = ["Limpiar el baño", "Ir de compras", "Ordenar", "Cortar el césped"];
tareas.slice(1);      // ["Ir de compras", "Ordenar", "Cortar el césped"]
tareas.slice(0, 2);   // ["Limpiar el baño", "Ir de compras"] → el fin no se incluye
tareas.slice(2, 4);   // ["Ordenar", "Cortar el césped"]
const copia = tareas.slice(); // copia completa
console.log(tareas.length);  // 4 → el original sigue intacto
```

splice() · cirugía

Modifica el array y devuelve lo que ha quitado.

slice() · fotocopia

No modifica el array: devuelve una copia del tramo.

05 · COPIAR PARTE DE UN ARRAY

Los índices negativos cuentan desde el final

	Limpiar el baño	Ir de compras	Ordenar	Cortar el césped
positivo	[0]	[1]	[2]	[3]
negativo	[-4]	[-3]	[-2]	[-1]

`tareas.slice(-2)` → `["Ordenar", "Cortar el césped"]`

`tareas.slice(1, -1)` → `["Ir de compras", "Ordenar"]`

`tareas.slice(1, -2)` → `["Ir de compras"]`

`tareas.slice(1, -3)` → `[]`

reverse() invierte; sort() ordena... como si fuera texto

```
const nombres = ["Juan", "Marta", "Pedro"];
nombres.reverse(); // ["Pedro", "Marta", "Juan"]

const letras = ["c", "a", "b"];
letras.sort(); // ["a", "b", "c"]

const numeros = [10, 9, 1, 25];
numeros.sort(); // [1, 10, 25, 9] ⚠
```

⚠ La trampa de sort()

Sin función de comparación convierte los elementos a texto: «10» va antes que «9» porque «1» < «9».

Ambos modifican el array

Reordenan el original y además lo devuelven.

Una función de comparación decide el orden

```
function comparar(a, b) {  
  if (a < b) return -1; // a va antes  
  if (a > b) return 1;  // a va después  
  return 0;            // iguales  
}  
  
const valores = [7, 6, 4, 8, 7, 2, 4];  
valores.sort(comparar); // [2, 4, 4, 6, 7, 7, 8]  
// Versión corta con función flecha (ES6)  
valores.sort((a, b) => a - b); // ascendente  
valores.sort((a, b) => b - a); // descendente
```

Lo que devuelve la función:

< 0 a va antes que b

0 mantienen su orden

> 0 b va antes que a

Pásala **sin paréntesis**: `sort(comparar)`, no `sort(comparar())`

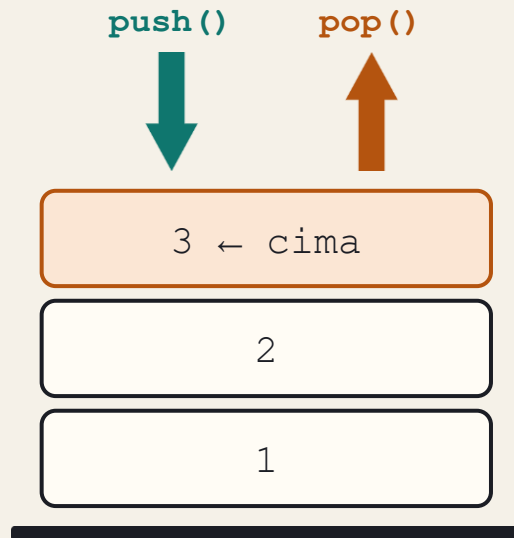
Ordenar objetos por una propiedad con `localeCompare()`

```
const contactos = [
  { nombre: "Juan",  apellido: "Díaz",    email: "juan@correo.es" },
  { nombre: "Ana",   apellido: "Ruiz",     email: "zeta@correo.es" },
  { nombre: "Pedro", apellido: "Álvarez",  email: "pedro@correo.es" }
];

const porNombre = (c1, c2) => c1.nombre.localeCompare(c2.nombre);
const porApellido = (c1, c2) => c1.apellido.localeCompare(c2.apellido);
contactos.sort(porNombre);    // Ana, Juan, Pedro
contactos.sort(porApellido);  // Pedro (Álvarez), Juan (Díaz), Ana (Ruiz)
```

`localeCompare()` respeta el idioma: «Álvarez» va antes que «Díaz». Comparando con `<` y `>`, la «Á» quedaría detrás de la «Z».

Pila (LIFO): el último en entrar es el primero en salir



```
const pila = [];  
pila.push(1);  
pila.push(2);  
pila.push(3);  
pila.pop(); // 3 → el último que entró  
  
// Uso real: el botón «Deshacer»  
const historial = [];  
historial.push("escribir texto");  
historial.push("poner negrita");  
historial.pop(); // deshace "poner negrita"
```

Cola (FIFO): el primero en entrar es el primero en salir



```
const cola = [];  
cola.push("Cliente 1");  
cola.push("Cliente 2", "Cliente 3");  
cola.shift();      // "Cliente 1" → el primero que llegó  
cola.shift();      // "Cliente 2"  
console.log(cola); // ["Cliente 3"]
```

Alternativa equivalente: **unshift()** para entrar y **pop()** para salir.

Encapsula la pila en un módulo ES6 y úsala con import

```
pila.js

export const crearPila = () => {
  const elementos = []; // privado
  return {
    apilar: (valor) => elementos.push(valor),
    desapilar: () => elementos.pop(),
    cima: () => elementos[elementos.length - 1],
    estaVacia: () => elementos.length === 0
  };
};
```

```
app.js

import { crearPila } from "./pila.js";

const deshacer = crearPila();
deshacer.apilar("escribir");
deshacer.apilar("borrar");
deshacer.desapilar(); // "borrar"
```

```
<script type="module" src="app.js">
```

Ejercicio: crea **cola.js** con `encolar()`, `desencolar()` y `estaVacia()`, e impórtalo en `app.js`.

09 · BUSCAR ELEMENTOS

indexOf() y lastIndexOf() devuelven la posición (o -1)

-20	500.5	-40	-34.5	200	500.5	-20	200
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

```
const transacciones = [-20, 500.5, -40, -34.5, 200, 500.5, -20, 200];
transacciones.indexOf(200);           // 4   (primera aparición)
transacciones.lastIndexOf(200);       // 7   (última aparición)
transacciones.indexOf(200, 5);         // 7   (empieza a buscar en el 5)
transacciones.lastIndexOf(200, 5);     // 4   (hacia atrás desde el 5)
transacciones.indexOf(500);           // -1  (no existe)
```

09 · BUSCAR ELEMENTOS

includes() dice si existe; find() busca con un criterio

```
includes() · ES2016 → devuelve true / false
```

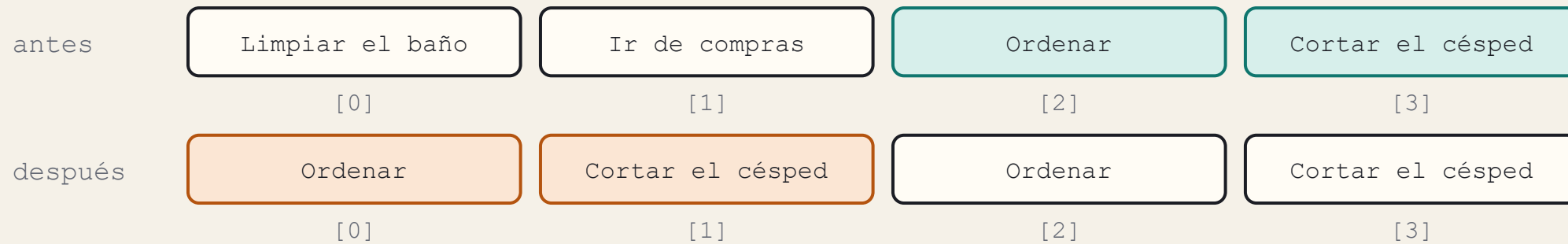
```
[4, 5, 6].includes(4);           // true
["Ana", "Luis"].includes("Eva"); // false
[4, 5, 6, 50].includes(6, 3);    // false (busca desde el índice 3)
```

```
find() · findIndex() · ES6 → reciben una función
```

```
const numeros = [2, 3, 4, 5, 6, 7];
const esImpar = (n) => n % 2 !== 0;
numeros.find(esImpar);           // 3 → el primer elemento que cumple
numeros.findIndex(esImpar);      // 1 → su índice
numeros.find((n) => n > 10);      // undefined si ninguno cumple
numeros.findIndex((n) => n > 10); // -1
```

10 · COPIAR DENTRO DEL ARRAY

copyWithin(destino, inicio, fin) sobrescribe con una copia interna



```
const tareas = ["Limpiar el baño", "Ir de compras", "Ordenar", "Cortar el césped"];
tareas.copyWithin(0, 2, 4);
// pega en la posición 0 una copia de los índices 2 y 3 (el 4 no se incluye)
console.log(tareas.length); // 4 → la longitud no cambia
```

11 · CONVERTIR A CADENA

toString(), toLocaleString() y join() convierten el array en texto

```
const datos = ["María", "Juan", 1234567.89];
datos.toString();           // "María,Juan,1234567.89"
datos.toLocaleString("es-ES"); // "María,Juan,1.234.567,89"
datos.toLocaleString("en-US"); // "María,Juan,1,234,567.89"
datos.join(" - ");          // "María - Juan - 1234567.89"
`Datos: ${datos}`;          // plantilla ES6: usa toString()
```

toString()

Separa siempre con comas.

toLocaleString()

Formatea números y fechas según el idioma.

join(separador)

Tú eliges el separador.

RESUMEN

¿Modifica el array original?

Sí, lo modifica

`push()` `unshift()` `pop()` `shift()` `splice()`
`sort()` `reverse()` `copyWithin()`

No, devuelve algo nuevo

`slice()` `concat()` `indexOf()` `lastIndexOf()`
`includes()` `find()` `findIndex()` `join()`
`toString()`

```
const copia = [...original]; // ES6: copia antes si necesitas conservar el original
copia.sort((a, b) => a - b);
```